# CHAPTER 14
# Exploring JavaScript

JavaScript brings a dynamic functionality to your websites. Every time you see something pop up when you mouse over an item in the browser, or see new text, colors, or images appear on the page in front of your eyes, or grab an object on the page and drag it to a new location—these things are generally done through JavaScript (although CSS is getting more and more powerful and can do many of these things too). It offers effects that are not otherwise possible, because it runs inside the browser and has direct access to all the elements in a web document.

JavaScript first appeared in the Netscape Navigator browser in 1995, coinciding with the addition of support for Java technology in the browser. Because of the initial incorrect impression that JavaScript was a spin-off of Java, there has been some long-term confusion over their relationship. However, the naming was just a marketing ploy to help the new scripting language benefit from the popularity of the Java programming language.

JavaScript gained new power when the HTML elements of the web page got a more formal, structured definition in what is called the *Document Object Model* (DOM). The DOM makes it relatively easy to add a new paragraph or focus on a piece of text and change it.

Because both JavaScript and PHP support much of the structured programming syntax used by the C programming language, they look very similar to each other. They are both fairly high-level languages, too. Also, they are weakly typed, so it's easy to change a variable to a new type just by using it in a new context.

Now that you have learned PHP, you should find JavaScript even easier. And you'll be glad you did, because it's at the heart of the asynchronous communication technology that provides the fluid web frontends that (along with HTML5 features) savvy web users expect these days.

# JavaScript and HTML Text

JavaScript is a client-side scripting language that runs entirely inside the web browser or under *Node.js*. To call it up, you place it between opening `<script>` and closing `</script>` HTML tags. A typical "Hello World" document using JavaScript might look like Example 14-1.

*Example 14-1. "Hello World" displayed using JavaScript*

```
<html>
  <head><title>Hello World</title></head>
  <body>
    <script type="text/javascript">
      document.write("Hello World")
    </script>
    <noscript>
      Your browser doesn't support or has disabled JavaScript
    </noscript>
  </body>
</html>
```

You may have seen web pages that use the HTML tag `<script language="javascript">`, but that usage has now been deprecated. This example uses the more recent and preferred `<script type="text/javascript">`, or you can just use `<script>` on its own if you like.

Within the `<script>` tags is a single line of JavaScript code that uses its equivalent of the PHP `echo` or `print` commands, `document.write`. As you'd expect, it simply outputs the supplied string to the current document, where it is displayed.

You may also have noticed that, unlike with PHP, there is no trailing semicolon (`;`). This is because a newline serves the same purpose as a semicolon in JavaScript. However, if you wish to have more than one statement on a single line, you do need to place a semicolon after each command except the last one. Of course, if you wish, you can add a semicolon to the end of every statement, and your JavaScript will work fine. My personal preference is to leave out the semicolon because it's superfluous, and I therefore also steer clear of practices that could cause issues. At the end of the day, though, the choice may come down to the team you work in, which more often than not may require semicolons, just to be sure. So, if in doubt, just add the semicolons.

The other thing to note in this example is the `<noscript>` and `</noscript>` pair of tags. These are used when you wish to offer alternative HTML to users whose browsers do not support JavaScript or who have it disabled. Using these tags is up to you, as they are not required, but you really ought to use them because it's usually not that

difficult to provide static HTML alternatives to the operations you provide using JavaScript. However, the remaining examples in this book will omit `<noscript>` tags, because we're focusing on what you can do with JavaScript, not what you can do without it.

When Example 14-1 is loaded, a web browser with JavaScript enabled will output the following (see Figure 14-1):

```
Hello World
```

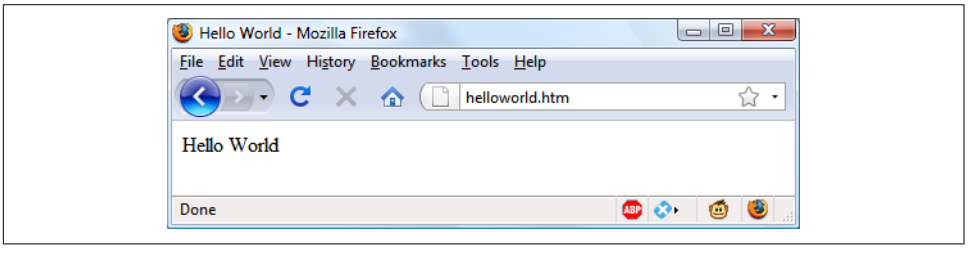


*Figure 14-1. JavaScript, enabled and working*

A browser with JavaScript disabled will display this message (see Figure 14-2):

```
Your browser doesn't support or has disabled JavaScript
```

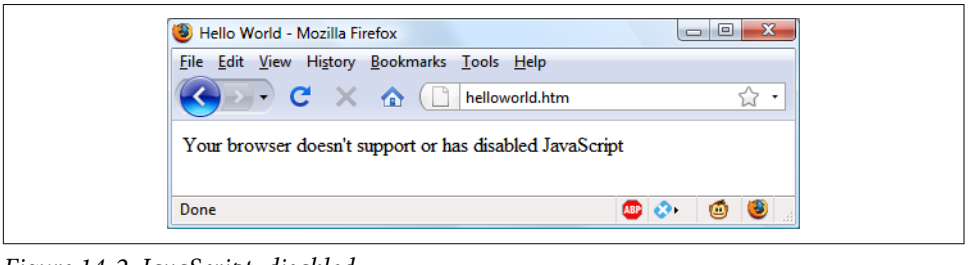


*Figure 14-2. JavaScript, disabled*

## Using Scripts Within a Document Head

In addition to placing a script within the body of a document, you can put it in the `<head>` section, which is the ideal place if you wish to execute a script when a page loads. If you place critical code and functions there, you can also ensure that they are ready to use immediately by any other script sections in the document that rely on them.

Another reason for placing a script in the document head is to enable JavaScript to write things such as meta tags into the `<head>` section, because the location of your script is the part of the document it writes to by default.

## Older and Nonstandard Browsers

If you need to support browsers that do not offer scripting (very unlikely in this day and age), you will need to use the HTML comment tags (`<!--` and `-->`) to prevent them from encountering script code that they should not see. Example 14-2 shows how you add them to your script code.

*Example 14-2. The "Hello World" example modified for non-JavaScript browsers*

```
<html>
  <head><title>Hello World</title></head>
  <body>
    <script type="text/javascript"><!--
      document.write("Hello World")
    // -->
    </script>
  </body>
</html>
```

Here an opening HTML comment tag (`<!--`) has been added directly after the opening `<script>` statement, and a closing comment tag (`// -->`) has been added directly before the script is closed with `</script>`.

The double forward slash (`//`) is used by JavaScript to indicate that the rest of the line is a comment. It is there so that browsers that *do* support JavaScript will ignore the following `-->`, but non-JavaScript browsers will ignore the preceding `//` and act on the `-->` by closing the HTML comment.

Although the solution is a little convoluted, all you really need to remember is to use the two following lines to enclose your JavaScript when you wish to support very old or nonstandard browsers:

```
<script type="text/javascript"><!--
  (Your JavaScript goes here...)
// -->
</script>
```

However, the use of these comments is unnecessary for any browser released over the past several years, but you do need to be aware of this, just in case.

## Including JavaScript Files

In addition to writing JavaScript code directly in HTML documents, you can include files of JavaScript code either from your website or from anywhere on the internet. The syntax for this is as follows:

```
<script type="text/javascript" src="script.js"></script>
```

Or, to pull a file in from the internet, use this (here without the `type="text/javascript"` as it is optional):

```
<script src="http://someserver.com/script.js"></script>
```

As for the script files themselves, they must *not* include any `<script>` or `</script>` tags, because they are unnecessary: the browser already knows that a JavaScript file is being loaded. Putting them in the JavaScript files will cause an error.

Including script files is the preferred way for you to use third-party JavaScript files on your website.

It is possible to leave out the `type="text/javascript"` parameter; all modern browsers default to assuming that the script contains JavaScript.

## Debugging JavaScript Errors

When you're learning JavaScript, it's important to be able to track typing or other coding errors. Unlike PHP, which displays error messages in the browser, JavaScript handles error messages in a way that changes according to the browser used. Table 14-1 lists how to access JavaScript error messages in the most commonly used browsers.

*Table 14-1. Accessing JavaScript error messages in different browsers*

| Browser | How to access JavaScript error messages |
|---|---|
| Apple Safari | Open Safari and choose Safari > Preferences > Advanced. Then select Show Develop menu in menu bar. Choose Develop > Show Error Console. |
| Google Chrome, Microsoft Edge, Mozilla Firefox, & Opera | Press Ctrl-Shift-J on a PC or Command-Shift-J on a Mac. |

Please refer to the browser developers' documentation on their websites for full details on using them.

# Using Comments

Because of their shared inheritance from the C programming language, PHP and JavaScript have many similarities, one of which is commenting. First, there's the single-line comment, like this:

```
// This is a comment
```

This style uses a pair of forward slash characters (`//`) to inform JavaScript that everything that follows is to be ignored. You also have multiline comments, like this:

```
/* This is a section
   of multiline comments
   that will not be
   interpreted */
```

You start a multiline comment with the sequence `/*` and end it with `*/`. Just remember that you cannot nest multiline comments, so make sure that you don't comment out large sections of code that already contain multiline comments.

## Semicolons

Unlike PHP, JavaScript generally does not require semicolons if you have only one statement on a line. Therefore, the following is valid:

```
x += 10
```

However, when you wish to place more than one statement on a line, you must separate them with semicolons, like this:

```
x += 10; y -= 5; z = 0
```

You can normally leave the final semicolon off, because the newline terminates the final statement.

There are exceptions to the semicolon rule. If you write JavaScript bookmarklets, or end a statement with a variable or function reference, *and* the first character of the line below is a left parenthesis or bracket, you *must* remember to append a semicolon or the JavaScript will fail. So, when in doubt, use a semicolon.

## Variables

No particular character identifies a variable in JavaScript as the dollar sign does in PHP. Instead, variables use the following naming rules:

- A variable may include only the letters `a-z`, `A-Z`, `0-9`, the `$` symbol, and the underscore (_).
- No other characters, such as spaces or punctuation, are allowed in a variable name.
- The first character of a variable name can be only `a-z`, `A-Z`, `$`, or _ (no numbers).
- Names are case-sensitive. `Count`, `count`, and `COUNT` are all different variables.
- There is no set limit on variable name lengths.

And yes, you're right: a `$` is there in that list of allowed characters. It *is* allowed by JavaScript and *may* be the first character of a variable or function name. Although I

don't recommend keeping the $ characters, this rule lets you port a lot of PHP code more quickly to JavaScript.

## String Variables

JavaScript string variables should be enclosed in either single or double quotation marks, like this:

```
greeting = "Hello there"
warning  = 'Be careful'
```

You may include a single quote within a double-quoted string or a double quote within a single-quoted string. But you must escape a quote of the same type by using the backslash character, like this:

```
greeting = "\"Hello there\" is a greeting"
warning  = '\'Be careful\' is a warning'
```

To read from a string variable, you can assign it to another one, like this:

```
newstring = oldstring
```

or you can use it in a function, like this:

```
status = "All systems are working"
document.write(status)
```

## Numeric Variables

Creating a numeric variable is as simple as assigning a value, like these examples:

```
count       = 42
temperature = 98.4
```

Like strings, numeric variables can be read from and used in expressions and functions.

## Arrays

JavaScript arrays are also very similar to those in PHP, in that an array can contain string or numeric data, as well as other arrays. To assign values to an array, use the following syntax (which in this case creates an array of strings):

```
toys = ['bat', 'ball', 'whistle', 'puzzle', 'doll']
```

To create a multidimensional array, nest smaller arrays within a larger one. So, to create a two-dimensional array containing the colors of a single face of a scrambled Rubik's Cube (where the colors red, green, orange, yellow, blue, and white are represented by their capitalized initial letters), you could use the following code:

```
face =
[
```

```
    ['R', 'G', 'Y'],
    ['W', 'R', 'O'],
    ['Y', 'W', 'G']
]
```

The previous example has been formatted to make it obvious what is going on, but it could also be written like this:

```
face = [['R', 'G', 'Y'], ['W', 'R', 'O'], ['Y', 'W', 'G']]
```

or even like this:

```
top = ['R', 'G', 'Y']
mid = ['W', 'R', 'O']
bot = ['Y', 'W', 'G']

face = [top, mid, bot]
```

To access the element two down and three along in this matrix, you would use the following (because array elements start at position 0):

```
document.write(face[1][2])
```

This statement will output the letter `O` for *orange*.

JavaScript arrays are powerful storage structures, so Chapter 16 discusses them in much greater depth.

# Operators

Operators in JavaScript, as in PHP, can involve mathematics, changes to strings, and comparison and logical operations (`and`, `or`, etc.). JavaScript mathematical operators look a lot like plain arithmetic—for instance, the following statement outputs `15`:

```
document.write(13 + 2)
```

The following sections teach you about the various operators.

## Arithmetic Operators

*Arithmetic operators* are used to perform mathematics. You can use them for the main four operations (addition, subtraction, multiplication, and division) as well as to find the modulus (the remainder after a division) and to increment or decrement a value (see Table 14-2).

*Table 14-2. Arithmetic operators*

| Operator | Description | Example |
|---|---|---|
| + | Addition | `j + 12` |
| - | Subtraction | `j - 22` |
| * | Multiplication | `j * 7` |
| / | Division | `j / 3.13` |
| % | Modulus (division remainder) | `j % 6` |
| ++ | Increment | `++j` |
| -- | Decrement | `--j` |

## Assignment Operators

The *assignment operators* are used to assign values to variables. They start with the very simple = and move on to +=, -=, and so on. The operator += adds the value on the right side to the variable on the left, instead of totally replacing the value on the left. Thus, if `count` starts with the value 6, the statement:

```
count += 1
```

sets `count` to 7, just like the more familiar assignment statement:

```
count = count + 1
```

Table 14-3 lists the various assignment operators available.

*Table 14-3. Assignment operators*

| Operator | Example | Equivalent to |
|---|---|---|
| = | `j = 99` | `j = 99` |
| += | `j += 2` | `j = j + 2` |
| += | `j += 'string'` | `j = j + 'string'` |
| -= | `j -= 12` | `j = j - 12` |
| *= | `j *= 2` | `j = j * 2` |
| /= | `j /= 6` | `j = j / 6` |
| %= | `j %= 7` | `j = j % 7` |

## Comparison Operators

*Comparison operators* are generally used inside a construct such as an `if` statement, where you need to compare two items. For example, you may wish to know whether a variable you have been incrementing has reached a specific value, or whether another variable is less than a set value, and so on (see Table 14-4).

*Table 14-4. Comparison operators*

| Operator | Description | Example |
|---|---|---|
| == | Is *equal* to | j == 42 |
| != | Is *not equal* to | j != 17 |
| > | Is *greater than* | j > 0 |
| < | Is *less than* | j < 100 |
| >= | Is *greater than or equal* to | j >= 23 |
| <= | Is *less than or equal* to | j <= 13 |
| === | Is *equal* to (and of the same type) | j === 56 |
| !== | Is *not equal* to (and of the same type) | j !== '1' |

## Logical Operators

Unlike PHP, JavaScript's *logical operators* do not include `and` and `or` equivalents to `&&` and `||`, and there is no `xor` operator (see Table 14-5).

*Table 14-5. Logical operators*

| Operator | Description | Example |
|---|---|---|
| && | And | j == 1 && k == 2 |
| \|\| | Or | j < 100 \|\| j > 0 |
| ! | Not | ! (j == k) |

## Incrementing, Decrementing, and Shorthand Assignment

The following forms of post- and pre-incrementing and decrementing that you learned to use in PHP are also supported by JavaScript, as are shorthand assignment operators:

```
++x
--y
x += 22
y -= 3
```

## String Concatenation

JavaScript handles string concatenation slightly differently from PHP. Instead of the `.` (period) operator, it uses the plus sign (+), like this:

```
document.write("You have " + messages + " messages.")
```

Assuming that the variable `messages` is set to the value 3, the output from this line of code will be as follows:

```
You have 3 messages.
```

Just as you can add a value to a numeric variable with the += operator, you can also append one string to another the same way:

```
name =  "James"
name += " Dean"
```

## Escape Characters

Escape characters, which you've seen used to insert quotation marks in strings, can also be used to insert various special characters such as tabs, newlines, and carriage returns. Here is an example using tabs to lay out a heading—it is included here merely to illustrate escapes, because in web pages, there are better ways to do layout:

```
heading = "Name\tAge\tLocation"
```

Table 14-6 details the escape characters available.

*Table 14-6. JavaScript's escape characters*

| Character | Meaning |
|---|---|
| \b | Backspace |
| \f | Form feed |
| \n | Newline |
| \r | Carriage return |
| \t | Tab |
| \' | Single quote (or apostrophe) |
| \" | Double quote |
| \\ | Backslash |
| *XXX* | An octal number between 000 and 377 that represents the Latin-1 character equivalent (such as \251 for the © symbol) |
| \x*XX* | A hexadecimal number between 00 and FF that represents the Latin-1 character equivalent (such as \xA9 for the © symbol) |
| \u*XXXX* | A hexadecimal number between 0000 and FFFF that represents the Unicode character equivalent (such as \u00A9 for the © symbol) |

# Variable Typing

Like PHP, JavaScript is a very loosely typed language; the *type* of a variable is determined only when a value is assigned and can change as the variable appears in different contexts. Usually, you don't have to worry about the type; JavaScript figures out what you want and just does it.

Take a look at Example 14-3, in which:

1. The variable n is assigned the string value '838102050'. The next line prints out its value, and the typeof operator is used to look up the type.

2. `n` is given the value returned when the numbers `12345` and `67890` are multiplied together. This value is also `838102050`, but it is a number, not a string. The type of the variable is then looked up and displayed.
3. Some text is appended to the number `n` and the result is displayed.

*Example 14-3. Setting a variable's type by assignment*

```
<script>
  n = '838102050'        // Set 'n' to a string
  document.write('n = ' + n + ', and is a ' + typeof n + '<br>')

  n = 12345 * 67890;     // Set 'n' to a number
  document.write('n = ' + n + ', and is a ' + typeof n + '<br>')

  n += ' plus some text' // Change 'n' from a number to a string
  document.write('n = ' + n + ', and is a ' + typeof n + '<br>')
</script>
```

The output from this script looks like this:

```
n = 838102050, and is a string
n = 838102050, and is a number
n = 838102050 plus some text, and is a string
```

If there is ever any doubt about the type of a variable, or you need to ensure that a variable has a particular type, you can force it to that type by using statements such as the following (which, respectively, turn a string into a number and a number into a string):

```
n = "123"
n *= 1    // Convert 'n' into a number

n = 123
n += ""   // Convert 'n' into a string
```

Or you can use the following functions in the same way:

```
n = "123"
n = parseInt(n)   // Convert 'n' into an integer number
n = parseFloat(n) // Convert 'n' into a floating point number


n = 123
n = n.toString()  // Convert 'n' into a string
```

You can read more about type conversion in JavaScript online.. And you can always look up a variable's type by using the `typeof` operator.

# Functions

As with PHP, JavaScript functions are used to separate out sections of code that perform a particular task. To create a function, declare it in the manner shown in Example 14-4.

*Example 14-4. A simple function declaration*

```
<script>
  function product(a, b)
  {
    return a*b
  }
</script>
```

This function takes the two parameters passed, multiplies them together, and returns the product.

# Global Variables

*Global* variables are ones defined outside of any functions (or defined within functions but without the `var` keyword). They can be defined in the following ways:

```
      a = 123               // Global scope
  var b = 456               // Global scope
  if (a == 123) var c = 789 // Global scope
```

Regardless of whether you are using the `var` keyword, as long as a variable is defined outside of a function, it is global in scope. This means that every part of a script can have access to it.

# Local Variables

Parameters passed to a function automatically have *local* scope, that is, they can be referenced only from within that function. However, there is one exception. Arrays are passed to a function by reference, so if you modify any elements in an array parameter, the elements of the original array will be modified.

To define a local variable that has scope only within the current function, and has not been passed as a parameter, use the `var` keyword. Example 14-5 shows a function that creates one variable with global scope and two with local scope.

*Example 14-5. A function creating variables with global and local scope*

```
<script>
  function test()
```

```
  {
        a = 123               // Global scope
    var b = 456               // Local scope
    if (a == 123) var c = 789 // Local scope
  }
</script>
```

To test whether scope setting has worked in PHP, we can use the `isset` function. But in JavaScript there is no such function, so Example 14-6 makes use of the `typeof` operator, which returns the string `undefined` when a variable is not defined.

*Example 14-6. Checking the scope of the variables defined in the function test*

```
<script>
  test()

  if (typeof a != 'undefined') document.write('a = "' + a + '"<br>')
  if (typeof b != 'undefined') document.write('b = "' + b + '"<br>')
  if (typeof c != 'undefined') document.write('c = "' + c + '"<br>')

  function test()
  {
    a     = 123
    var b = 456

    if (a == 123) var c = 789
  }
</script>
```

The output from this script is the following single line:

```
a = "123"
```

This shows that only the variable `a` was given global scope, which is exactly what we would expect, since the variables `b` and `c` were given local scope by being prefaced with the `var` keyword.

If your browser issues a warning about `b` being undefined, the warning is correct but can be ignored.

## Using let and const

JavaScript now offers two new keywords: `let` and `const`. The `let` keyword is pretty much a swap-in for `var`, but it has the advantage that you cannot redeclare a variable once you have done so with `let`, although you can with `var`.

You see, the fact that you could redeclare variables using `var` was leading to obscure bugs, such as the following:

```
var hello   = "Hello there"
var counter = 1

if (counter > 0)
{
  var hello = "How are you?"
}

document.write(hello)
```

Can you see the problem? Because `counter` is greater than 0 (since we initialized it to 1), the string `hello` is redefined as "How are you?" which is then displayed in the document.

Now, if you replace the `var` with `let` (as follows), the second declaration is ignored, and the original string "Hello there" will be displayed:

```
let hello   = "Hello there"
let counter = 1

if (counter > 0)
{
  let hello = "How are you?"
}

document.write(hello)
```

The `var` keyword is either globally scoped (if outside of any blocks or functions) or *function* scoped, and variables declared with it are initialized with `undefined`, but the `let` keyword is either globally or *block* scoped, and variables are not initialized.

Any variable assigned using `let` has scope either within the entire document if declared outside of any block, or, if declared within a block bounded by `{}` (which includes functions), its scope is limited to that block (and any nested sub-blocks). If you declare a variable within a block but try to access it from outside that block, an error will be returned, as with the following, which will fail at the `document.write` because `hello` will have no value:

```
let counter = 1

if (counter > 0)
{
  let hello = "How are you?"
}

document.write(hello)
```

You can use `let` to declare variables of the same name as previously declared ones, as long as it is within a new scope, in which case any previous value assigned to a variable of the same name in the previous scope will become inaccessible to the new

scope, because the new variable of the same name is treated as totally different from the previous one. It only has scope within the current block, or any sub-blocks (unless another `let` is used to declare yet another variable of the same name in a sub-block).

It is good practice to try avoiding the reuse of meaningful variable names, or you risk causing confusion. However, loop or index variables such as `i` (or other short and simple names) can generally be reused in new scopes without causing any confusion.

You can further increase your control over scope by declaring a variable to have a constant value, that is, one that cannot be changed. This is beneficial where you have created a variable that you are treating as a constant but had declared it only using `var` or `let`, because you might have instances in your code where you try to change that value, which would be allowed but would be a bug.

However, if you use the `const` keyword to declare the variable and assign its value, any attempt to change the value later will be disallowed, and your code will halt with an error message in the console similar to:

```
Uncaught TypeError: Assignment to constant variable
```

The following code will cause just that error:

```
const hello = "Hello there"
let counter = 1

if (counter > 0)
{
  hello = "How are you?"
}

document.write(hello)
```

Just like `let`, `const` declarations are also block scoped (within `{}` sections and any sub-blocks), meaning that you can have constant variables of the same name but have different values in different scopes of a piece of code. However, I strongly recommend you try to avoid duplication of names and keep any constant name for one single value throughout each program, using a new constant name wherever you need a new constant.

In summary: `var` has global or function scope, and `let` and `const` have global or block scope. Both `var` and `let` can be declared without being initialized, while `const` must be initialized during declaration. The `var` keyword can be reused to re-declare a `var` variable, but `let` and `const` cannot. Finally, `const` can neither be redeclared nor reassigned.

You may prefer to use a developer console with tests such as these (and elsewhere in this book) as previously explained in "Debugging JavaScript Errors" on page 325, in which case you can replace `document.write` with `console.log`, and the output will be shown in the console instead of within the browser. This is also a better option for JavaScript that will run once a document has fully loaded, because at that time `document.write` would replace the current document, rather than append to it, which is probably not what you might intend.

# The Document Object Model

The design of JavaScript is very smart. Rather than just creating yet another scripting language (which would have still been a pretty good improvement at the time), there was a vision to build it around the already-existing HTML Document Object Model. This breaks down the parts of an HTML document into discrete *objects*, each with its own *properties* and *methods* and each subject to JavaScript's control.

JavaScript separates objects, properties, and methods by using a period (one good reason why `+` is the string concatenation operator in JavaScript, rather than the period). For example, let's consider a business card as an object we'll call `card`. This object contains properties such as a name, address, phone number, and so on. In the syntax of JavaScript, these properties would look like this:

```
card.name
card.phone
card.address
```

Its methods are functions that retrieve, change, and otherwise act on the properties. For instance, to invoke a method that displays the properties of the object `card`, you might use syntax such as this:

```
card.display()
```

Have a look at some of the earlier examples in this chapter and notice where the statement `document.write` is used. Now that you understand how JavaScript is based around objects, you will see that `write` is actually a method of the `document` object.

Within JavaScript, there is a hierarchy of parent and child objects, which is what is known as the Document Object Model (DOM; see Figure 14-3).

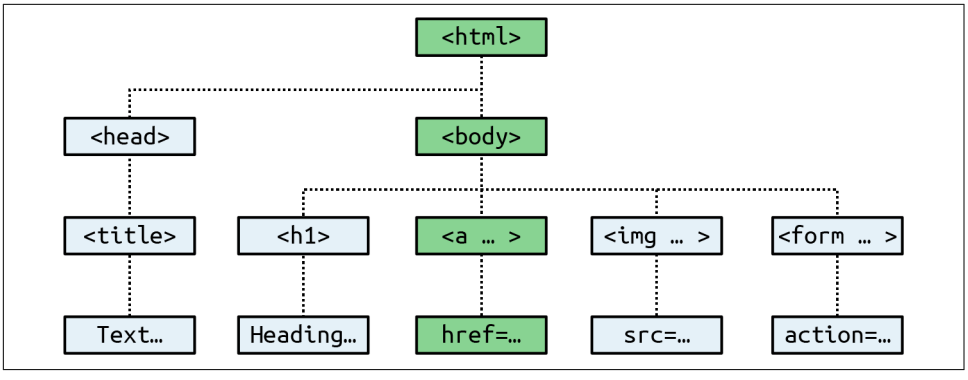


*Figure 14-3. Example of DOM object hierarchy*

The figure uses HTML tags that you are already familiar with to illustrate the parent/child relationship between the various objects in a document. For example, a URL within a link is part of the body of an HTML document. In JavaScript, it is referenced like this:

```
url = document.links.linkname.href
```

Notice how this follows the central column down. The first part, `document`, refers to the `<html>` and `<body>` tags; `links.linkname` refers to the `<a>` tag, and `href` to the `href` attribute.

Let's turn this into some HTML and a script to read a link's properties. Type Example 14-7 and save it as *linktest.html*; then call it up in your browser.

*Example 14-7. Reading a link URL with JavaScript*

```
<html>
  <head>
    <title>Link Test</title>
  </head>
  <body>
    <a id="mylink" href="http://mysite.com">Click me</a><br>
    <script>
      url = document.links.mylink.href
      document.write('The URL is ' + url)
    </script>
  </body>
</html>
```

Note the short form of the `<script>` tags, where I have omitted the parameter `type="text/JavaScript"` to save you some typing. If you wish, just for the purposes of testing this (and other examples), you could also omit everything outside of the `<script>` and `</script>` tags. The output from this example is as follows:

```
Click me
The URL is http://mysite.com
```

The second line of output comes from the `document.write` method. Notice how the code follows the document tree down from `document` to `links` to `mylink` (the `id` given to the link) to `href` (the URL destination value).

There is also a short form that works equally well, which starts with the value in the `id` attribute: `mylink.href`. So, you can replace this:

```
url = document.links.mylink.href
```

with the following:

```
url = mylink.href
```

## Another Use for the $ Symbol

As mentioned earlier, the `$` symbol is allowed in JavaScript variable and function names. Because of this, you may sometimes encounter strange-looking code like this:

```
url = $('mylink').href
```

Some enterprising programmers have decided that the `getElementById` function is so prevalent in JavaScript that they have written a function to replace it called `$`, like in jQuery (although jQuery uses the `$` for much more than that—see Chapter 22), as shown in Example 14-8.

*Example 14-8. A replacement function for the `getElementById` method*

```
<script>
  function $(id)
  {
    return document.getElementById(id)
  }
</script>
```

Therefore, as long as you have included the `$` function in your code, syntax such as this:

```
$('mylink').href
```

can replace code such as this:

```
document.getElementById('mylink').href
```

## Using the DOM

The `links` object is actually an array of URLs, so the `mylink` URL in Example 14-7 can also be safely referred to in all browsers in the following way (because it's the first, and only, link):

```
url = document.links[0].href
```

If you want to know how many links there are in an entire document, you can query the `length` property of the `links` object like this:

```
numlinks = document.links.length
```

You can extract and display all links in a document like this:

```
for (j=0 ; j < document.links.length ; ++j)
  document.write(document.links[j].href + '<br>')
```

The `length` of something is a property of every array, and many objects as well. For example, the number of items in your browser's web history can be queried like this:

```
document.write(history.length)
```

To stop websites from snooping on your browsing history, the `history` object stores only the number of sites in the array: you cannot read from or write to these values. But you can replace the current page with one from the history, if you know what position it has within the history. This can be very useful in cases in which you know that certain pages in the history came from your site, or you simply wish to send the browser back one or more pages, which you do with the `go` method of the `history` object. For example, to send the browser back three pages, issue the following command:

```
history.go(-3)
```

You can also use the following methods to move back or forward a page at a time:

```
history.back()
history.forward()
```

In a similar manner, you can replace the currently loaded URL with one of your choosing, like this:

```
document.location.href = 'http://google.com'
```

Of course, there's a whole lot more to the DOM than reading and modifying links. As you progress through the following chapters on JavaScript, you'll become quite familiar with the DOM and how to access it.

# About document.write

When teaching programming, it's necessary to have a quick and easy way to display the results of expressions. In PHP (for example) there are the `echo` and `print` statements, which simply send text to the browser, so that's easy. In JavaScript, though, there are the following alternatives.

## Using console.log

The `console.log` function will output the result of any value or expression passed to it in the console of the current browser. This is a special mode with a frame or window separate from the browser window, and in which errors and other messages can be made to display. While great for experienced programmers, it is not ideal for beginners because the output is not near the web content in the browser.

## Using alert

The `alert` function displays values or expressions passed to it in a pop-up window, which requires you to click a button to close. Clearly this can become quite irritating very quickly, and it has the downside of displaying only the current message—previous ones are erased.

## Writing into Elements

It is possible to write directly into the text of an HTML element, which is a fairly elegant solution (and the best one for production websites)—except that for this book every example would require such an element to be created, and some lines of code to access it. This gets in the way of teaching the core of an example and would make the code look overly cumbersome and confusing.

## Using document.write

The `document.write` function writes a value or expression at the current browser location and is therefore the perfect choice for quickly displaying results. It keeps all the examples short and sweet, by placing the output right there in the browser next to the web content and code.

You may, however, have heard that this function is regarded as unsafe by some developers, because when you call it after a web page is fully loaded, it will overwrite the current document. While this is correct, it doesn't apply to any of the examples in this book, because they all use `document.write` the way it was originally intended: as part of the page creation process, calling it only before the page has completed loading and displaying.

However, although I use `document.write` in this way for simple examples, I never use it in production code (except in the very rarest of circumstances where it actually is necessary). Instead, I almost always use the preceding option of writing directly into a specially prepared element, per the more complex examples in Chapter 18 onward (which access the `innerHTML` property of elements for program output).

So, please remember that where you see `document.write` being called in this book, it is there only to simplify an example, and I recommend that you also use the function only in this same way—for obtaining quick test results.

With that caveat explained, in the following chapter we'll continue our exploration of JavaScript by looking at how to control program flow and write expressions.

## Questions

1. Which tags do you use to enclose JavaScript code?
2. By default, to which part of a document will JavaScript code output?
3. How can you include JavaScript code from another source in your documents?
4. Which JavaScript function is the equivalent of `echo` or `print` in PHP?
5. How can you create a comment in JavaScript?
6. What is the JavaScript string concatenation operator?
7. Which keyword can you use within a JavaScript function to define a variable that has local scope?
8. Give two cross-browser methods to display the URL assigned to the link with an `id` of `thislink`.
9. Which two JavaScript commands will make the browser load the previous page in its history array?
10. What JavaScript command would you use to replace the current document with the main page at the *oreilly.com* website?

See "Chapter 14 Answers" on page 758 in the Appendix for the answers to these questions.

CHAPTER 15

# Expressions and Control Flow in JavaScript

In the previous chapter, I introduced the basics of JavaScript and the DOM. Now it's time to look at how to construct complex expressions in JavaScript and how to control the program flow of your scripts by using conditional statements.

## Expressions

JavaScript expressions are very similar to those in PHP. As you learned in Chapter 4, an expression is a combination of values, variables, operators, and functions that results in a value; the result can be a number, a string, or a Boolean value (which evaluates to either `true` or `false`).

Example 15-1 shows some simple expressions. For each line, it prints out a letter between `a` and `d`, followed by a colon and the result of the expressions. The `<br>` tag is there to create a line break and separate the output into four lines (remember that both `<br>` and `<br />` are acceptable in HTML5, so I chose to use the former style for brevity).

*Example 15-1. Four simple Boolean expressions*

```
<script>
  document.write("a: " + (42 > 3) + "<br>")
  document.write("b: " + (91 < 4) + "<br>")
  document.write("c: " + (8 == 2) + "<br>")
  document.write("d: " + (4 < 17) + "<br>")
</script>
```

The output from this code is as follows:

```
a: true
b: false
```

```
c: false
d: true
```

Notice that both expressions `a:` and `d:` evaluate to `true`, but `b:` and `c:` evaluate to `false`. Unlike PHP (which would print the number `1` and nothing, respectively), the actual strings `true` and `false` are displayed.

In JavaScript, when you are checking whether a value is `true` or `false`, all values evaluate to `true` except the following, which evaluate to `false`: the string `false` itself, `0`, `-0`, the empty string, `null`, `undefined`, and `NaN` (Not a Number, a computer engineering concept for the result of an illegal floating-point operation such as division by zero).

Note that I am referring to `true` and `false` in lowercase. This is because, unlike in PHP, these values *must* be in lowercase in JavaScript. Therefore, only the first of the two following statements will display, printing the lowercase word `true`, because the second will cause a `'TRUE' is not defined` error:

```
if (1 == true) document.write('true') // True
if (1 == TRUE) document.write('TRUE') // Will cause an error
```

Remember that any code snippets you wish to type and try for yourself in an HTML file need to be enclosed within `<script>` and `</script>` tags.

# Literals and Variables

The simplest form of an expression is a *literal*, which means something that evaluates to itself, such as the number `22` or the string `Press Enter`. An expression could also be a variable, which evaluates to the value that has been assigned to it. They are both types of expressions, because they return a value.

Example 15-2 shows three different literals and two variables, all of which return values, albeit of different types.

*Example 15-2. Five types of literals*

```
<script>
  myname = "Peter"
  myage  = 24
  document.write("a: " + 42     + "<br>") // Numeric literal
  document.write("b: " + "Hi"   + "<br>") // String literal
  document.write("c: " + true   + "<br>") // Constant literal
  document.write("d: " + myname + "<br>") // String variable
  document.write("e: " + myage  + "<br>") // Numeric variable
</script>
```

And, as you'd expect, you see a return value from all of these in the following output:

```
a: 42
b: Hi
c: true
d: Peter
e: 24
```

Operators let you create more complex expressions that evaluate to useful results. When you combine assignment or control-flow constructs with expressions, the result is a *statement*.

Example 15-3 shows one of each. The first assigns the result of the expression `366 - day_number` to the variable `days_to_new_year`, and the second outputs a friendly message only if the expression `days_to_new_year < 30` evaluates to `true`.

*Example 15-3. Two simple JavaScript statements*

```
<script>
  day_number       = 127  // For example
  days_to_new_year = 366 - day_number
  if (days_to_new_year < 30) document.write("It's nearly New Year")
  else                       document.write("It's a long time to go")
</script>
```

# Operators

JavaScript offers a lot of powerful operators, ranging from arithmetic, string, and logical operators to assignment, comparison, and more (see Table 15-1).

*Table 15-1. JavaScript operator types*

| Operator | Description | Example |
|---|---|---|
| Arithmetic | Basic mathematics | `a + b` |
| Array | Array manipulation | `a + b` |
| Assignment | Assign values | `a = b + 23` |
| Bitwise | Manipulate bits within bytes | `12 ^ 9` |
| Comparison | Compare two values | `a < b` |
| Increment/decrement | Add or subtract one | `a++` |
| Logical | Boolean | `a && b` |
| String | Concatenation | `a + 'string'` |

Each operator takes a different number of operands:

- *Unary* operators, such as incrementing (`a++`) or negation (`-a`), take a single operand.
- *Binary* operators, which represent the bulk of JavaScript operators—including addition, subtraction, multiplication, and division—take two operands.
- The one *ternary* operator, which takes the form `? x : y`, requires three operands. It's a terse single-line `if` statement that chooses between two expressions depending on a third one.

## Operator Precedence

Like PHP, JavaScript utilizes operator precedence, in which some operators in an expression are processed before others and are therefore evaluated first. Table 15-2 lists JavaScript's operators and their precedences.

*Table 15-2. Precedence of JavaScript operators (high to low)*

| Operator(s) | Type(s) |
|---|---|
| `() [] .` | Parentheses, call, and member |
| `++ --` | Increment/decrement |
| `+ - ~ !` | Unary, bitwise, and logical |
| `* / %` | Arithmetic |
| `+ -` | Arithmetic and string |
| `<< >> >>>` | Bitwise |
| `< > <= >=` | Comparison |
| `== != === !==` | Comparison |
| `& ^ \|` | Bitwise |
| `&&` | Logical |
| `\|\|` | Logical |
| `? :` | Ternary |
| `= += -= *= /= %=` | Assignment |
| `<<= >>= >>>= &= ^= \|=` | Assignment |
| `,` | Separator |

## Associativity

Most JavaScript operators are processed in order from left to right in an equation. But some operators require processing from right to left instead. The direction of processing is called the operator's *associativity*.

This associativity becomes important where you do not explicitly force precedence (which you should always do, by the way, because it makes code more readable and

less error prone). For example, look at the following assignment operators, by which three variables are all set to the value `0`:

```
level = score = time = 0
```

This multiple assignment is possible only because the rightmost part of the expression is evaluated first and then processing continues in a right-to-left direction. Table 15-3 lists the JavaScript operators and their associativity.

*Table 15-3. Operators and associativity*

| Operator | Description | Associativity |
|---|---|---|
| `++ --` | Increment and decrement | None |
| `new` | Create a new object | Right |
| `+ - ~ !` | Unary and bitwise | Right |
| `?:` | Ternary | Right |
| `= *= /= %= += -=` | Assignment | Right |
| `<<= >>= >>>= &= ^= \|=` | Assignment | Right |
| `,` | Separator | Left |
| `+ - * / %` | Arithmetic | Left |
| `<< >> >>>` | Bitwise | Left |
| `< <= > >= == != === !==` | Arithmetic | Left |

## Relational Operators

*Relational operators* test two operands and return a Boolean result of either `true` or `false`. There are three types of relational operators: *equality*, *comparison*, and *logical*.

### Equality operators

The equality operator is `==` (which should not be confused with the `=` assignment operator). In Example 15-4, the first statement assigns a value, and the second tests it for equality. As it stands, nothing will be printed out, because `month` is assigned the string value `July`, and therefore the check for it having a value of `October` will fail.

*Example 15-4. Assigning a value and testing for equality*

```
<script>
  month = "July"
  if (month == "October") document.write("It's the Fall")
</script>
```

If the two operands of an equality expression are of different types, JavaScript will convert them to whatever type makes best sense to it. For example, any strings composed entirely of numbers will be converted to numbers whenever compared with a

number. In Example 15-5, a and b are two different values (one is a number, and the other is a string), and we would therefore normally expect neither of the if statements to output a result.

*Example 15-5. The equality and identity operators*

```
<script>
  a = 3.1415927
  b = "3.1415927"
  if (a == b)  document.write("1")
  if (a === b) document.write("2")
</script>
```

However, if you run the example, you will see that it outputs the number 1, which means that the first if statement evaluated to true. This is because the string value of b was temporarily converted to a number, and therefore both halves of the equation had a numerical value of 3.1415927.

In contrast, the second if statement uses the *identity* operator, three equals signs in a row, which prevents JavaScript from automatically converting types. a and b are therefore found to be different, so nothing is output.

As with forcing operator precedence, whenever you're in doubt about how JavaScript will convert operand types, you can use the identity operator to turn this behavior off.

### Comparison operators

Using comparison operators, you can test for more than just equality and inequality. JavaScript also gives you > (is greater than), < (is less than), >= (is greater than or equal to), and <= (is less than or equal to) to play with. Example 15-6 shows these operators in use.

*Example 15-6. The four comparison operators*

```
<script>
  a = 7; b = 11
  if (a > b)  document.write("a is greater than b<br>")
  if (a < b)  document.write("a is less than b<br>")
  if (a >= b) document.write("a is greater than or equal to b<br>")
  if (a <= b) document.write("a is less than or equal to b<br>")
</script>
```

In this example, where a is 7 and b is 11, the following is output (because 7 is less than 11 and also less than or equal to 11):

```
a is less than b
a is less than or equal to b
```

### Logical operators

Logical operators produce true or false results and are also known as *Boolean* operators. There are three of them in JavaScript (see Table 15-4).

*Table 15-4. JavaScript's logical operators*

| Logical operator | Description |
|---|---|
| `&&` *(and)* | `true` if both operands are `true` |
| `\|\|` *(or)* | `true` if either operand is `true` |
| `!` *(not)* | `true` if the operand is `false`, or `false` if the operand is `true` |

You can see how these can be used in Example 15-7, which outputs `0`, `1`, and `true`.

*Example 15-7. The logical operators in use*

```
<script>
  a = 1; b = 0
  document.write((a && b) + "<br>")
  document.write((a || b) + "<br>")
  document.write((  !b  ) + "<br>")
</script>
```

The `&&` statement requires both operands to be `true` to return a value of `true`, the `||` statement will be `true` if either value is `true`, and the third statement performs a `NOT` on the value of `b`, turning it from `0` into a value of `true`.

The `||` operator can cause unintentional problems, because the second operand will not be evaluated if the first is evaluated as `true`. In Example 15-8, the `getnext` function will never be called if `finished` has a value of `1` (these are purely examples, and the action of `getnext` is irrelevant to this explanation—just think of it as a function that does *something* when called).

*Example 15-8. A statement using the `||` operator*

```
<script>
  if (finished == 1 || getnext() == 1) done = 1
</script>
```

If you *need* `getnext` to be called at each `if` statement, you should rewrite the code as shown in Example 15-9.

*Example 15-9. The `if...or` statement modified to ensure calling of `getnext`*

```
<script>
  gn = getnext()
```

```
  if (finished == 1 OR gn == 1) done = 1;
</script>
```

In this case, the code in the function `getnext` will be executed and its return value stored in `gn` before the `if` statement.

Table 15-5 shows all the possible variations of using the logical operators. You should also note that `!true` equals `false` and `!false` equals `true`.

*Table 15-5. All possible logical expressions*

| Inputs | | Operators and results | |
|---|---|---|---|
| a | b | && | \|\| |
| true | true | true | true |
| true | false | false | true |
| false | true | false | true |
| false | false | false | false |

# The with Statement

The `with` statement is not one that you've seen in the earlier chapters on PHP, because it's exclusive to JavaScript, and also one that while you need to know it, you should not use (see page 351). With it (if you see what I mean), you can simplify some types of JavaScript statements by reducing many references to an object to just one reference. References to properties and methods within the `with` block are assumed to apply to that object.

For example, take the code in Example 15-10, in which the `document.write` function never references the variable `string` by name.

*Example 15-10. Using the `with` statement*

```
<script>
  string = "The quick brown fox jumps over the lazy dog"

  with (string)
  {
    document.write("The string is " + length + " characters<br>")
    document.write("In upper case it's: " + toUpperCase())
  }
</script>
```

Even though `string` is never directly referenced by `document.write`, this code still manages to output the following:

```
The string is 43 characters
In upper case it's: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
```

```
    // Use a different method to create an XMLHTTPRequest object
  }
</script>
```

There's also another keyword associated with `try` and `catch` called `finally` that is always executed, regardless of whether an error occurs in the `try` clause. To use it, just add something like the following statements after a `catch` statement:

```
finally
{
  alert("The 'try' clause was encountered")
}
```

# Conditionals

Conditionals alter program flow. They enable you to ask questions about certain things and respond to the answers you get in different ways. There are three types of nonlooping conditionals: the `if` statement, the `switch` statement, and the `?` operator.

## The if Statement

Several examples in this chapter have already made use of `if` statements. The code within such a statement is executed only if the given expression evaluates to `true`. Multiline `if` statements require curly braces around them, but as in PHP, you can omit the braces for single statements, although it's often a good idea to use them anyway, especially when writing code in which the number of actions within an `if` statement might change as development proceeds. Therefore, the following statements are valid:

```
if (a > 100)
{
  b=2
  document.write("a is greater than 100")
}

if (b == 10) document.write("b is equal to 10")
```

## The else Statement

When a condition has not been met, you can execute an alternative by using an `else` statement, like this:

```
if (a > 100)
{
  document.write("a is greater than 100")
}
else
{
```

```
  document.write("a is less than or equal to 100")
}
```

Unlike PHP, JavaScript has no `elseif` statement, but that's not a problem because you can use an `else` followed by another `if` to form the equivalent of an `elseif` statement, like this:

```
if (a > 100)
{
  document.write("a is greater than 100")
}
else if(a < 100)
{
  document.write("a is less than 100")
}
else
{
  document.write("a is equal to 100")
}
```

As you can see, you can use another `else` after the new `if`, which could equally be followed by another `if` statement, and so on. Although I have shown braces on the statements, because each is a single line, the previous example could be written as follows:

```
if     (a > 100) document.write("a is greater than 100")
else if(a < 100) document.write("a is less than 100")
else             document.write("a is equal to 100")
```

## The switch Statement

The `switch` statement is useful when one variable or the result of an expression can have multiple values and you want to perform a different function for each value.

For example, the following code takes the PHP menu system we put together in Chapter 4 and converts it to JavaScript. It works by passing a single string to the main menu code according to what the user requests. Let's say the options are Home, About, News, Login, and Links, and we set the variable `page` to one of these according to the user's input.

The code for this written using `if...else if...` might look like Example 15-13.

*Example 15-13. A multiline `if...else if...` statement*

```
<script>
  if      (page == "Home")  document.write("You selected Home")
  else if (page == "About") document.write("You selected About")
  else if (page == "News")  document.write("You selected News")
  else if (page == "Login") document.write("You selected Login")
```

```
  else if (page == "Links") document.write("You selected Links")
</script>
```

But using a `switch` construct, the code could look like Example 15-14.

*Example 15-14. A `switch` construct*

```
<script>
  switch (page)
  {
    case "Home":
      document.write("You selected Home")
      break
    case "About":
      document.write("You selected About")
      break
    case "News":
      document.write("You selected News")
      break
    case "Login":
      document.write("You selected Login")
      break
    case "Links":
      document.write("You selected Links")
      break
  }
</script>
```

The variable `page` is mentioned only once at the start of the `switch` statement. Thereafter, the `case` command checks for matches. When one occurs, the matching conditional statement is executed. Of course, a real program would have code here to display or jump to a page, rather than simply telling the user what was selected.

You may also supply multiple cases for a single action. For example:

```
switch (heroName)
{
  case "Superman":
  case "Batman":
  case "Wonder Woman":
    document.write("Justice League")
    break
  case "Iron Man":
  case "Captain America":
  case "Spiderman":
    document.write("The Avengers")
    break
}
```

### Breaking out

As you can see in Example 15-14, just as with PHP, the `break` command allows your code to break out of the `switch` statement once a condition has been satisfied. Remember to include the `break` unless you want to continue executing the statements under the next `case`.

### Default action

When no condition is satisfied, you can specify a default action for a `switch` statement by using the `default` keyword. Example 15-15 shows a code snippet that could be inserted into Example 15-14.

*Example 15-15. A default statement to add to Example 15-14*

```
default:
  document.write("Unrecognized selection")
  break
```

## The ? Operator

The `ternary` operator (`?`), combined with the `:` character, provides a quick way of doing `if...else` tests. With it you can write an expression to evaluate and then follow it with a `?` symbol and the code to execute if the expression is `true`. After that, place a `:` and the code to execute if the expression evaluates to `false`.

Example 15-16 shows the ternary operator being used to print out whether the variable `a` is less than or equal to 5 and prints something either way.

*Example 15-16. Using the ternary operator*

```
<script>
  document.write(
    a <= 5 ?
    "a is less than or equal to 5" :
    "a is greater than 5"
  )
</script>
```

The statement has been broken up into several lines for clarity, but you would be more likely to use such a statement on a single line, in this manner:

```
    size = a <= 5 ? "short" : "long"
```

# Looping

Again, you will find many close similarities between JavaScript and PHP when it comes to looping. Both languages support `while`, `do...while`, and `for` loops.

## while Loops

A JavaScript `while` loop first checks the value of an expression and starts executing the statements within the loop only if that expression is `true`. If it is `false`, execution skips over to the next JavaScript statement (if any).

Upon completing an iteration of the loop, the expression is again tested to see if it is `true`, and the process continues until such a time as the expression evaluates to `false` or until execution is otherwise halted. Example 15-17 shows such a loop.

*Example 15-17. A `while` loop*

```
<script>
  counter=0

  while (counter < 5)
  {
    document.write("Counter: " + counter + "<br>")
    ++counter
  }
</script>
```

This script outputs the following:

```
Counter: 0
Counter: 1
Counter: 2
Counter: 3
Counter: 4
```

If the variable `counter` were not incremented within the loop, it is quite possible that some browsers could become unresponsive due to a never-ending loop, and the page may not even be easy to terminate with Escape or the Stop button. So, be careful with your JavaScript loops.

## do...while Loops

When you require a loop to iterate at least once before any tests are made, use a `do...while` loop, which is similar to a `while` loop, except that the test expression is checked only after each iteration of the loop. So, to output the first seven results in the 7 times table, you could use code such as that in Example 15-18.

*Example 15-18. A `do...while` loop*

```
<script>
  count = 1

  do
  {
    document.write(count + " times 7 is " + count * 7 + "<br>")
  } while (++count <= 7)
</script>
```

As you might expect, this loop outputs the following:

```
1 times 7 is 7
2 times 7 is 14
3 times 7 is 21
4 times 7 is 28
5 times 7 is 35
6 times 7 is 42
7 times 7 is 49
```

## for Loops

A `for` loop combines the best of all worlds into a single looping construct that allows you to pass three parameters for each statement:

- An initialization expression
- A condition expression
- A modification expression

These are separated by semicolons, like this: `for (expr1 ; expr2 ; expr3)`. The initialization expression is executed at the start of the first iteration of the loop. In the case of the code for the multiplication table for 7, `count` would be initialized to the value `1`. Then, each time around the loop, the condition expression (in this case, `count <= 7`) is tested, and the loop is entered only if the condition is `true`. Finally, at the end of each iteration, the modification expression is executed. In the case of the multiplication table for 7, the variable `count` is incremented. Example 15-19 shows what the code would look like.

*Example 15-19. Using a `for` loop*

```
<script>
  for (count = 1 ; count <= 7 ; ++count)
  {
    document.write(count + "times 7 is " + count * 7 + "<br>");
  }
</script>
```

As in PHP, you can assign multiple variables in the first parameter of a `for` loop by separating them with a comma, like this:

```
for (i = 1, j = 1 ; i < 10 ; i++)
```

Likewise, you can perform multiple modifications in the last parameter, like this:

```
for (i = 1 ; i < 10 ; i++, --j)
```

Or you can do both at the same time:

```
for (i = 1, j = 1 ; i < 10 ; i++, --j)
```

## Breaking Out of a Loop

The `break` command, which you'll recall is important inside a `switch` statement, is also available within `for` loops. You might need to use this, for example, when searching for a match of some kind. Once the match is found, you know that continuing to search will only waste time and make your visitor wait. Example 15-20 shows how to use the `break` command.

*Example 15-20. Using the `break` command in a `for` loop*

```
<script>
  haystack     = new Array()
  haystack[17] = "Needle"

  for (j = 0 ; j < 20 ; ++j)
  {
    if (haystack[j] == "Needle")
    {
      document.write("<br>- Found at location " + j)
      break
    }
    else document.write(j + ", ")
  }
</script>
```

This script outputs the following:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
- Found at location 17
```

## The continue Statement

Sometimes you don't want to entirely exit from a loop but instead wish to skip the remaining statements just for this iteration of the loop. In such cases, you can use the `continue` command. Example 15-21 shows this in use.

*Example 15-21. Using the `continue` command in a `for` loop*

```
<script>
  haystack     = new Array()
  haystack[4]  = "Needle"
  haystack[11] = "Needle"
  haystack[17] = "Needle"

  for (j = 0 ; j < 20 ; ++j)
  {
    if (haystack[j] == "Needle")
    {
      document.write("<br>- Found at location " + j + "<br>")
      continue
    }

    document.write(j + ", ")
  }
</script>
```

Notice how the second `document.write` call does not have to be enclosed in an `else` statement (as it did before), because the `continue` command will skip it if a match has been found. The output from this script is as follows:

```
0, 1, 2, 3,
- Found at location 4
5, 6, 7, 8, 9, 10,
- Found at location 11
12, 13, 14, 15, 16,
- Found at location 17
18, 19,
```

# Explicit Casting

Unlike PHP, JavaScript has no explicit casting of types such as `(int)` or `(float)`. Instead, when you need a value to be of a certain type, use one of JavaScript's built-in functions, shown in Table 15-6.

*Table 15-6. JavaScript's type-changing functions*

| Change to type | Function to use |
|---|---|
| `Int`, `Integer` | `parseInt()` |
| `Bool`, `Boolean` | `Boolean()` |
| `Float`, `Double`, `Real` | `parseFloat()` |
| `String` | `String()` |
| `Array` | `split()` |

So, for example, to change a floating-point number to an integer, you could use code such as the following (which displays the value 3):

```
n = 3.1415927
i = parseInt(n)
document.write(i)
```

Or you can use the compound form:

```
document.write(parseInt(3.1415927))
```

That's it for control flow and expressions. The next chapter focuses on the use of functions, objects, and arrays in JavaScript.

# Questions

1. How are Boolean values handled differently by PHP and JavaScript?
2. What characters are used to define a JavaScript variable name?
3. What is the difference between unary, binary, and ternary operators?
4. What is the best way to force your own operator precedence?
5. When would you use the === (identity) operator?
6. What are the simplest two forms of expressions?
7. Name the three conditional statement types.
8. How do `if` and `while` statements interpret conditional expressions of different data types?
9. Why is a `for` loop more powerful than a `while` loop?
10. What is the purpose of the `with` statement?

See "Chapter 15 Answers" on page 758 in the Appendix for the answers to these questions.

CHAPTER 16

# JavaScript Functions, Objects, and Arrays

Just like PHP, JavaScript offers access to functions and objects. In fact, JavaScript is actually based on objects, because—as you've seen—it has to access the DOM, which makes every element of an HTML document available to manipulate as an object.

The usage and syntax are also quite similar to those of PHP, so you should feel right at home as I take you through using functions and objects in JavaScript, as well as through an in-depth exploration of array handling.

## JavaScript Functions

In addition to having access to dozens of built-in functions (or methods), such as `write`, which you have already seen being used in `document.write`, you can easily create your own functions. Whenever you have a relatively complex piece of code that is likely to be reused, you have a candidate for a function.

### Defining a Function

The general syntax for a function is shown here:

```
function function_name([parameter [, ...]])
{
  statements
}
```

The first line of the syntax indicates the following:

- A definition starts with the word `function`.
- A name follows that must start with a letter or underscore, followed by any number of letters, digits, dollar signs, or underscores.

- The parentheses are required.
- One or more parameters, separated by commas, are optional (indicated by the square brackets, which are not part of the function syntax).

Function names are case-sensitive, so all of the following strings refer to different functions: `getInput`, `GETINPUT`, and `getinput`.

In JavaScript there is a general naming convention for functions: the first letter of each word in a name is capitalized, except for the very first letter, which is lowercase. Therefore, of the previous examples, `getInput` would be the preferred name used by most programmers. This convention is commonly referred to as *bumpyCaps*, *bumpy-Case*, or (most frequently) *camelCase*.

The opening curly brace starts the statements that will execute when you call the function; a matching curly brace must close it. These statements may include one or more `return` statements, which force the function to cease execution and return to the calling code. If a value is attached to the `return` statement, the calling code can retrieve it.

### The `arguments` array

The `arguments` array is a member of every function. With it, you can determine the number of variables passed to a function and what they are. Take the example of a function called `displayItems`. Example 16-1 shows one way of writing it.

*Example 16-1. Defining a function*

```
<script>
  displayItems("Dog", "Cat", "Pony", "Hamster", "Tortoise")

  function displayItems(v1, v2, v3, v4, v5)
  {
    document.write(v1 + "<br>")
    document.write(v2 + "<br>")
    document.write(v3 + "<br>")
    document.write(v4 + "<br>")
    document.write(v5 + "<br>")
  }
</script>
```

When you call up this script in your browser, it will display the following:

```
Dog
Cat
Pony
Hamster
Tortoise
```

All of this is fine, but what if you wanted to pass more than five items to the function? Also, reusing the `document.write` call multiple times instead of employing a loop is wasteful programming. Luckily, the `arguments` array gives you the flexibility to handle a variable number of arguments. Example 16-2 shows how you can use it to rewrite the previous example in a much more efficient manner.

*Example 16-2. Modifying the function to use the `arguments` array*

```
<script>
  let c = "Car"

  displayItems("Bananas", 32.3, c)

  function displayItems()
  {
    for (j = 0 ; j < displayItems.arguments.length ; ++j)
      document.write(displayItems.arguments[j] + "<br>")
  }
</script>
```

Note the use of the `length` property, which you already encountered in the previous chapter, and also that I reference the array `displayItems.arguments` using the variable `j` as an offset into it. I also chose to keep the function short and sweet by not surrounding the contents of the `for` loop in curly braces, as it contains only a single statement. Remember that the loop must stop when `j` is one less than `length`, not equal to `length`.

Using this technique, you now have a function that can take as many (or as few) arguments as you like and act on each argument as you desire.

## Returning a Value

Functions are not used just to display things. In fact, they are mostly used to perform calculations or data manipulations and then return a result. The function `fixNames` in Example 16-3 uses the `arguments` array (discussed in the previous section) to take a series of strings passed to it and return them as a single string. The "fix" it performs is to convert every character in the arguments to lowercase except for the first character of each argument, which is set to a capital letter.

*Example 16-3. Cleaning up a full name*

```
<script>
  document.write(fixNames("the", "DALLAS", "CowBoys"))

  function fixNames()
  {
```

```
    var s = ""

    for (j = 0 ; j < fixNames.arguments.length ; ++j)
      s += fixNames.arguments[j].charAt(0).toUpperCase() +
           fixNames.arguments[j].substr(1).toLowerCase() + " "

    return s.substr(0, s.length-1)
  }
</script>
```

When called with the parameters `the`, `DALLAS`, and `CowBoys`, for example, the function returns the string `The Dallas Cowboys`. Let's walk through the function.

It first initializes the temporary (and local) variable `s` to the empty string. Then a `for` loop iterates through each of the passed parameters, isolating the parameter's first character using the `charAt` method and converting it to uppercase with the `toUpperCase` method. The various methods shown in this example are all built into JavaScript and available by default.

Then the `substr` method is used to fetch the rest of each string, which is converted to lowercase via the `toLowerCase` method. A fuller version of the `substr` method here would specify how many characters are part of the substring as a second argument:

```
substr(1, (arguments[j].length) - 1 )
```

In other words, this `substr` method says, "Start with the character at position 1 (the second character) and return the rest of the string (the length minus one)." As a nice touch, though, the `substr` method assumes that you want the rest of the string if you omit the second argument.

After the whole argument is converted to our desired case, a space character is added to the end, and the result is appended to the temporary variable `s`.

Finally, the `substr` method is used again to return the contents of the variable `s`, except for the final space—which is unwanted. We remove this by using `substr` to return the string up to, but not including, the final character.

This example is particularly interesting in that it illustrates the use of multiple properties and methods in a single expression, for example:

```
fixNames.arguments[j].substr(1).toLowerCase()
```

You have to interpret the statement by mentally dividing it into parts at the periods. JavaScript evaluates these elements of the statement from left to right as follows:

1. Start with the name of the function itself: `fixNames`.
2. Extract element `j` from the array `arguments` representing `fixNames` arguments.

CHAPTER 17

# JavaScript and PHP Validation and Error Handling

With your solid foundation in both PHP and JavaScript, it's time to bring these technologies together to create web forms that are as user-friendly as possible.

We'll be using PHP to create the forms and JavaScript to perform client-side validation to ensure that the data is as complete and correct as it can be before it is submitted. Final validation of the input will then be done by PHP, which will, if necessary, present the form again to the user for further modification.

In the process, this chapter will cover validation and regular expressions in both JavaScript and PHP.

## Validating User Input with JavaScript

JavaScript validation should be considered an assistance more to your users than to your websites because, as I have already stressed many times, you cannot trust any data submitted to your server, even if it has supposedly been validated with JavaScript. This is because hackers can quite easily simulate your web forms and submit any data of their choosing.

Another reason you cannot rely on JavaScript to perform all your input validation is that some users disable JavaScript, or use browsers that don't support it.

So, the best types of validation to do in JavaScript are checking that fields have content if they are not to be left empty, ensuring that email addresses conform to the proper format, and ensuring that values entered are within expected bounds.

## The validate.html Document (Part 1)

Let's begin with a general signup form, common on most sites that offer memberships or user registration. The inputs requested will be *forename*, *surname*, *username*, *password*, *age*, and *email address*. Example 17-1 provides a good template for such a form.

*Example 17-1. A form with JavaScript validation (part 1)*

```
<!DOCTYPE html>
<html>
  <head>
    <title>An Example Form</title>
    <style>
      .signup {
        border:1px solid #999999;
        font:  normal 14px helvetica;
        color: #444444;
      }
    </style>
    <script>
      function validate(form)
      {
        fail  = validateForename(form.forename.value)
        fail += validateSurname(form.surname.value)
        fail += validateUsername(form.username.value)
        fail += validatePassword(form.password.value)
        fail += validateAge(form.age.value)
        fail += validateEmail(form.email.value)

        if   (fail == "")   return true
        else { alert(fail); return false }
      }
    </script>
  </head>
  <body>
    <table class="signup" border="0" cellpadding="2"
             cellspacing="5" bgcolor="#eeeeee">
      <th colspan="2" align="center">Signup Form</th>
      <form method="post" action="adduser.php" onsubmit="return validate(this)">
        <tr><td>Forename</td>
          <td><input type="text" maxlength="32" name="forename"></td></tr>
        <tr><td>Surname</td>
          <td><input type="text" maxlength="32" name="surname"></td></tr>
        <tr><td>Username</td>
          <td><input type="text" maxlength="16" name="username"></td></tr>
        <tr><td>Password</td>
          <td><input type="text" maxlength="12" name="password"></td></tr>
        <tr><td>Age</td>
          <td><input type="text" maxlength="3"  name="age"></td></tr>
        <tr><td>Email</td>
          <td><input type="text" maxlength="64" name="email"></td></tr>
```

```
        <tr><td colspan="2" align="center"><input type="submit"
          value="Signup"></td></tr>
      </form>
    </table>
  </body>
</html>
```

As it stands, this form will display correctly but will not self-validate, because the main validation functions have not yet been added. Even so, save it as *validate.html*, and when you call it up in your browser, it will look like Figure 17-1.

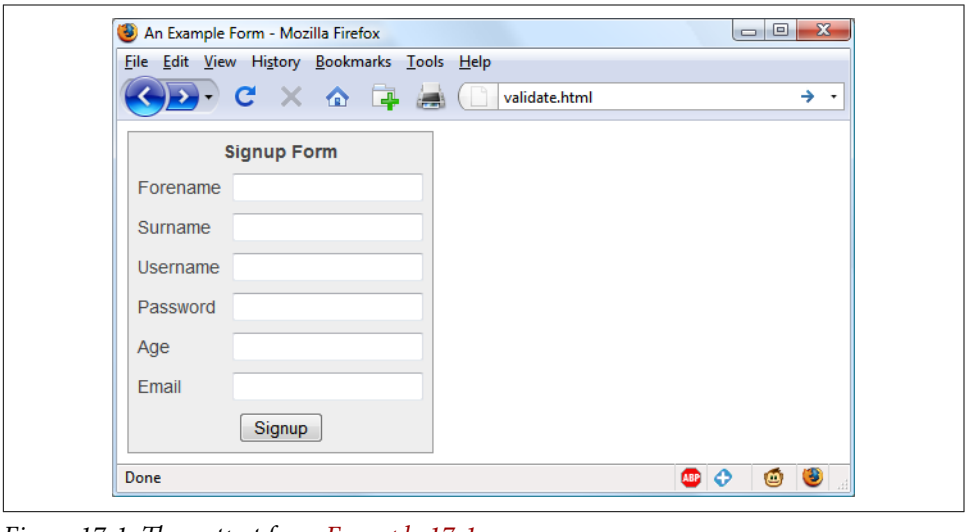


*Figure 17-1. The output from Example 17-1*

Let's look at how this document is made up. The first few lines set up the document and use a little CSS to make the form look a little less plain. The parts of the document related to JavaScript come next and are shown in bold.

Between the `<script>` and `</script>` tags lies a single function called `validate` that itself calls up six other functions to validate each of the form's input fields. We'll get to these functions shortly. For now I'll just explain that they return either an empty string if a field validates or an error message if it fails. If there are any errors, the final line of the script pops up an alert box to display them.

Upon passing validation, the `validate` function returns a value of `true`; otherwise, it returns `false`. The return values from `validate` are important, because if it returns `false`, the form is prevented from being submitted. This allows the user to close the alert pop-up and make changes. If `true` is returned, no errors were encountered in the form's fields, and so the form is submitted.

The second part of this example features the HTML for the form, with each field and its name placed within its own row of a table. This is pretty straightforward HTML, with the exception of the `onSubmit="return validate(this)"` statement within the opening `<form>` tag. Using `onSubmit`, you can cause a function of your choice to be called when a form is submitted. That function can perform some checking and return a value of either `true` or `false` to signify whether the form should be allowed to be submitted.

The `this` parameter is the current object (i.e., this form) and is passed to the `validate` function just discussed. The `validate` function receives this parameter as the object `form`.

As you can see, the only JavaScript used within the form's HTML is the call to `return` buried in the `onSubmit` attribute. Browsers with JavaScript disabled or not available will simply ignore the `onSubmit` attribute, and the HTML will display just fine.

## The validate.html Document (Part 2)

Now we come to Example 17-2, a set of six functions that do the actual form-field validation. I suggest that you type all of this second part and save it in the `<script>...</script>` section of Example 17-1, which you should already have saved as *validate.html*.

*Example 17-2. A form with JavaScript validation (part 2)*

```
function validateForename(field)
{
  return (field == "") ? "No Forename was entered.\n" : ""
}

function validateSurname(field)
{
  return (field == "") ? "No Surname was entered.\n" : ""
}

function validateUsername(field)
{
  if (field == "") return "No Username was entered.\n"
  else if (field.length < 5)
    return "Usernames must be at least 5 characters.\n"
  else if (/[^a-zA-Z0-9_-]/.test(field))
    return "Only a-z, A-Z, 0-9, - and _ allowed in Usernames.\n"
  return ""
}

function validatePassword(field)
{
  if (field == "") return "No Password was entered.\n"
```

```
  else if (field.length < 6)
    return "Passwords must be at least 6 characters.\n"
  else if (!/[a-z]/.test(field) || ! /[A-Z]/.test(field) ||
           !/[0-9]/.test(field))
    return "Passwords require one each of a-z, A-Z and 0-9.\n"
  return ""
}

function validateAge(field)
{
  if (field == "" || isNaN(field)) return "No Age was entered.\n"
  else if (field < 18 || field > 110)
    return "Age must be between 18 and 110.\n"
  return ""
}

function validateEmail(field)
{
  if (field == "") return "No Email was entered.\n"
    else if (!((field.indexOf(".") > 0) &&
               (field.indexOf("@") > 0)) ||
              /[^a-zA-Z0-9.@_-]/.test(field))
      return "The Email address is invalid.\n"
  return ""
}
```

We'll go through each of these functions in turn, starting with `validateForename`, so you can see how validation works.

### Validating the forename

`validateForename` is quite a short function that accepts the parameter `field`, which is the value of the forename passed to it by the `validate` function.

If this value is the empty string, an error message is returned; otherwise, an empty string is returned to signify that no error was encountered.

If the user entered spaces in this field, it would be accepted by `validateForename`, even though it's empty for all intents and purposes. You can fix this by adding an extra statement to trim whitespace from the field before checking whether it's empty, use a regular expression to make sure there's something besides whitespace in the field, or—as I do here—just let the user make the mistake and allow the PHP program to catch it on the server.

### Validating the surname

The `validateSurname` function is almost identical to `validateForename` in that an error is returned only if the surname supplied was an empty string. I chose not to

limit the characters allowed in either of the name fields to allow for possibilities such as non-English and accented characters.

### Validating the username

The `validateUsername` function is a little more interesting, because it has a more complicated job. It has to allow through only the characters `a-z`, `A-Z`, `0-9`, `_` and `-`, and ensure that usernames are at least five characters long.

The `if...else` statements commence by returning an error if `field` has not been filled in. If it's not the empty string, but is fewer than five characters in length, another error message is returned.

Then the JavaScript `test` function is called, passing a regular expression (which matches any character that is *not* one of those allowed) to be matched against `field` (see "Regular Expressions" on page 391). If even one character that isn't one of the acceptable characters is encountered, the `test` function returns `true`, and so `validateUser` returns an error string.

### Validating the password

Similar techniques are used in the `validatePassword` function. First the function checks whether `field` is empty, and if it is, it returns an error. Next, an error message is returned if the password is shorter than six characters.

One of the requirements we're imposing on passwords is that they must have at least one each of a lowercase, uppercase, and numerical character, so the `test` function is called three times, once for each of these cases. If any one of these calls returns `false`, one of the requirements was not met, and so an error message is returned. Otherwise, the empty string is returned to signify that the password was OK.

### Validating the age

`validateAge` returns an error message if `field` is not a number (determined by a call to the `isNaN` function) or if the age entered is lower than 18 or greater than 110. Your applications may well have different or no age requirements. Again, upon successful validation, the empty string is returned.

### Validating the email

In the last and most complicated example, the email address is validated with `validateEmail`. After checking whether anything was actually entered, and returning an error message if it wasn't, the function calls the JavaScript `indexOf` function twice. The first time a check is made to ensure there is a period (`.`) somewhere after the first character of the field, and the second checks that an `@` symbol appears somewhere after the first character.